Tower of Hanoi

The **Tower of Hanoi** (also called **The problem of Benares Temple** or **Tower of Brahma** or **Lucas' Tower** and sometimes pluralized as **Towers**, or simply **pyramid puzzle**) is a mathematical game or puzzle consisting of three rods and a number of disks of various diameters, which can slide onto any rod. The puzzle begins with the disks stacked on one rod in order of decreasing size, the smallest at the top, thus approximating a conical shape. The objective of the puzzle is to move the entire stack to the last rod, obeying the following rules:

1. Only one disk may be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
3. No disk may be placed on top of a disk that is smaller than it.

With 3 disks, the puzzle can be solved in 7 moves. The minimal number of moves required to solve a Tower of Hanoi puzzle is $2^n - 1$, where *n* is the number of disks.

The puzzle was invented by the French mathematician Édouard Lucas in 1883. Numerous myths regarding the ancient and mystical nature of the puzzle popped up almost immediately, including a myth about an Indian temple in Kashi Vishwanath containing a large room with three time-worn posts in it, surrounded by 64 golden disks. But, this story of Indian Kashi Vishwanath temple was spread tongue-in-cheek by a friend of Édouard Lucas

If the legend were true, and if the priests were able to move disks at a rate of one per second, using the smallest number of moves, it would take them $2^{64} - 1$ seconds or roughly 585 billion years to finish, which is about 42 times the current age of the universe.

Simpler statement of ITERATIVE solution

For an **EVEN** number of disks:

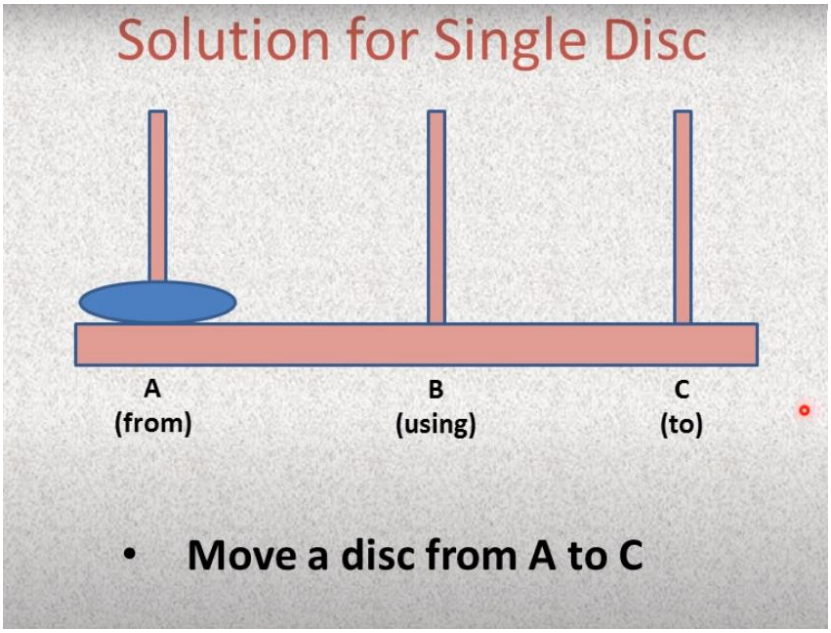
- Make the legal move between pegs A and B (in either direction),
- Make the legal move between pegs A and C (in either direction),
- Make the legal move between pegs B and C (in either direction),
- Repeat until complete.

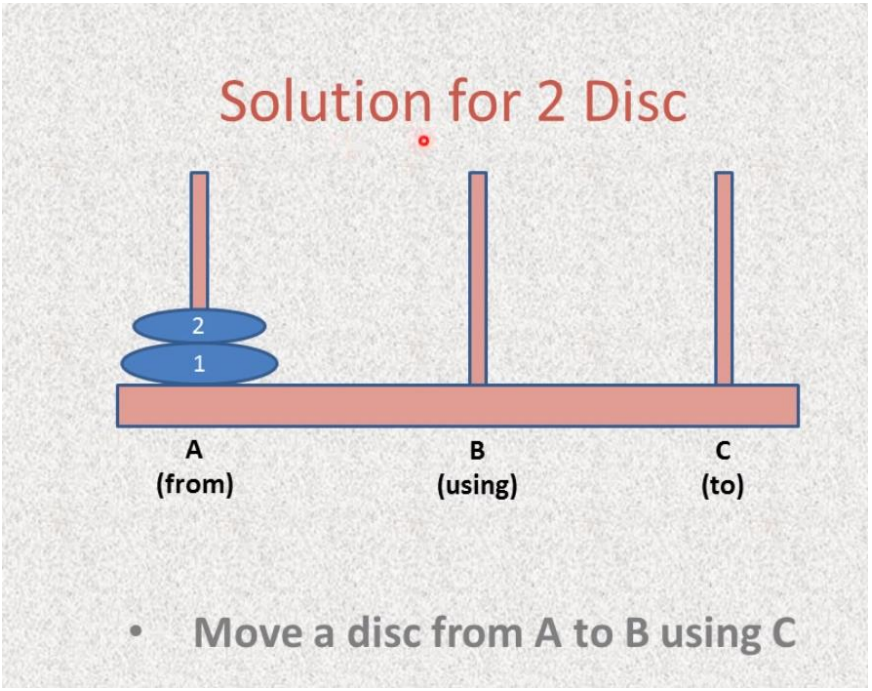
For an **ODD** number of disks:

- Make the legal move between pegs A and C (in either direction),
- Make the legal move between pegs A and B (in either direction),
- Make the legal move between pegs B and C (in either direction),
- Repeat until complete.

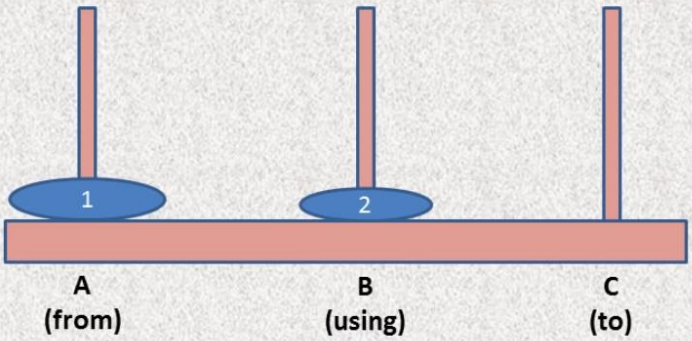
After each move check if the C peg is complete. In each case, a total of $2^n - 1$ moves are made.

Recursive Solution



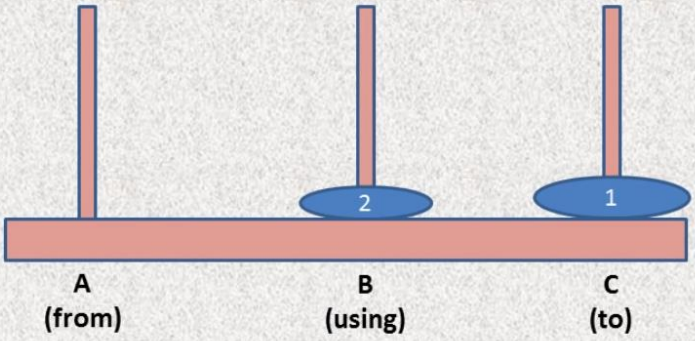


Solution for 2 Disc



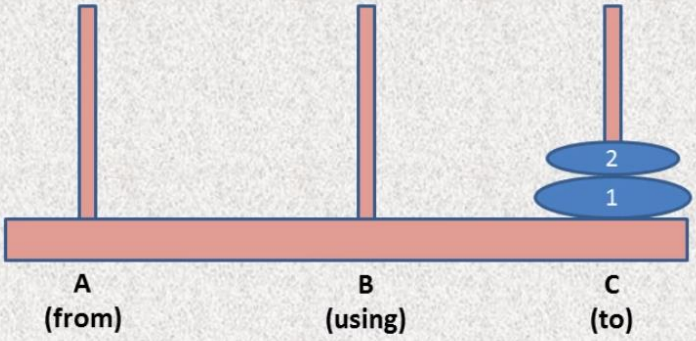
- **Move a disc from A to B using C**

Solution for 2 Disc



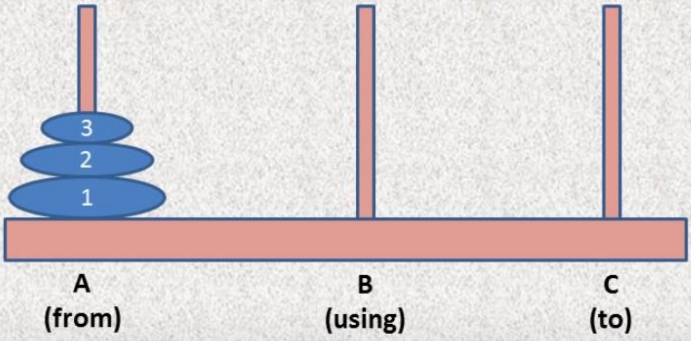
- **Move a Disc from A to B using C**
- **Move a Disc from A to C**

Solution for 2 Disc



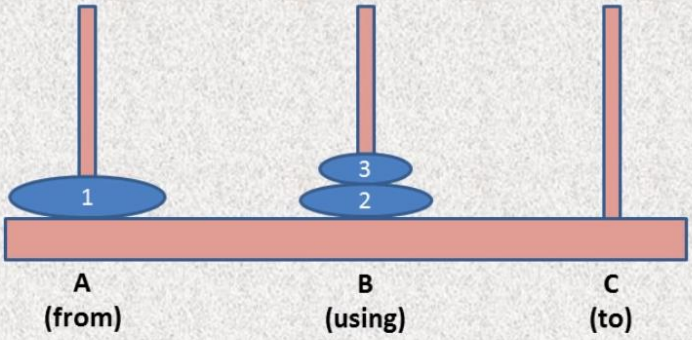
- **Move a Disc from A to B using C**
- **Move a Disc from A to C**
- **Move a Disc from B to C using A**

Solution for 3 Disc



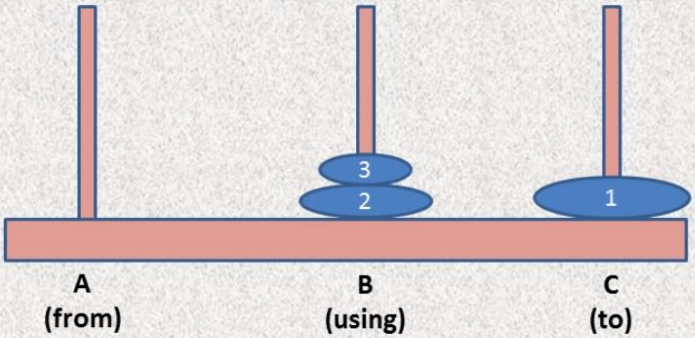
- **Move 2 Discs from A to B using C**

Solution for 3 Disc



- Move 2 Discs from A to B using C

Solution for 3 Disc

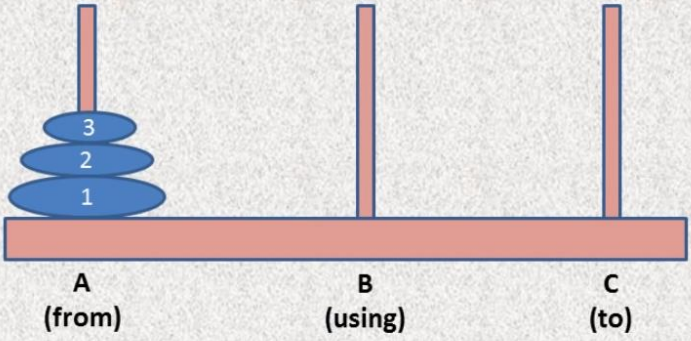


- Move 2 Discs from A to B using C
- Move a Disc from A to C
- Move 2 Discs from B to C using A

Solution for 3 Disc

- Move 2 Discs from A to B using C
- Move a Disc from A to C
- Move 2 Discs from B to C using A

Solution for n Disc



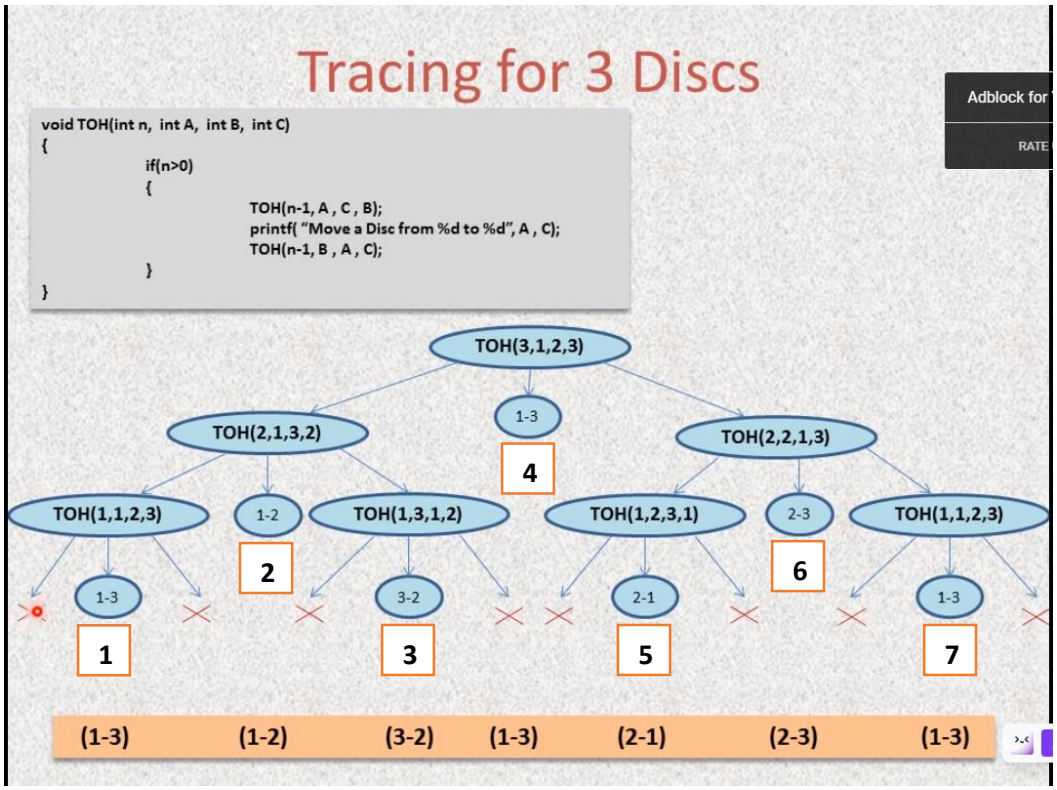
- Move n-1 Discs from A to B using C
- Move a Disc from A to C
- Move n-1 Discs from B to C using A

Program

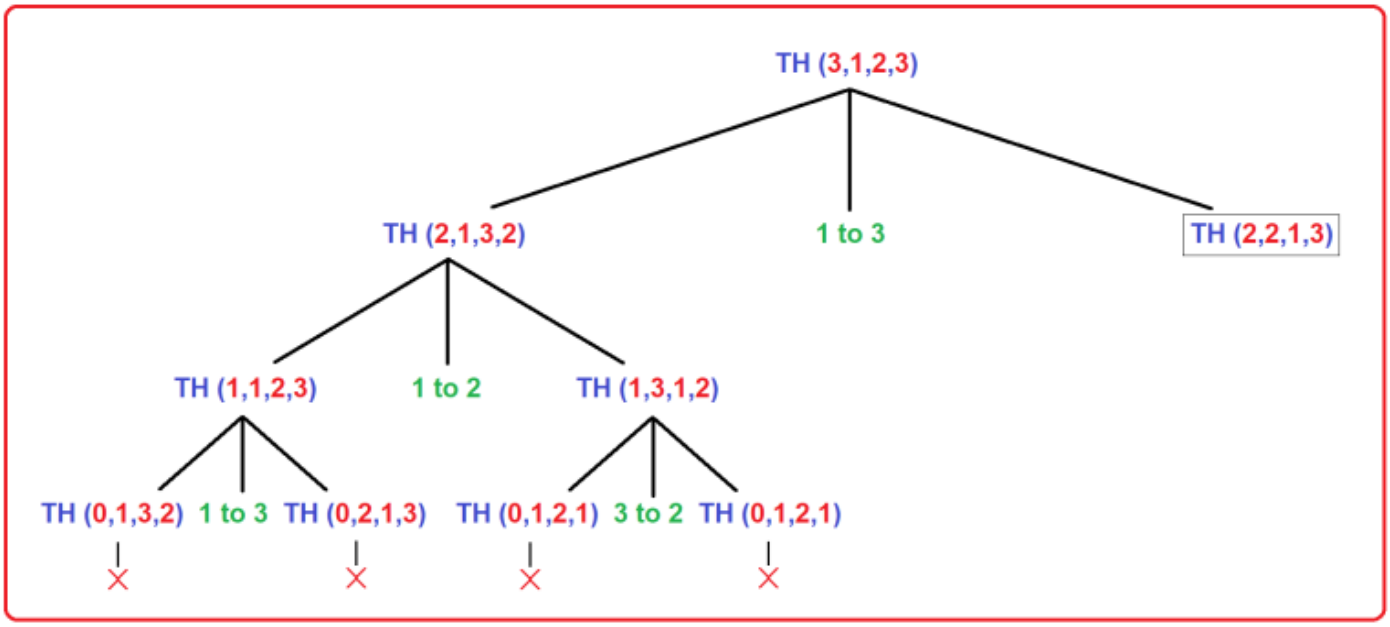
- Move n-1 Discs from A to B using C
- Move a Disc from A to C
- Move n-1 Discs from B to C using A

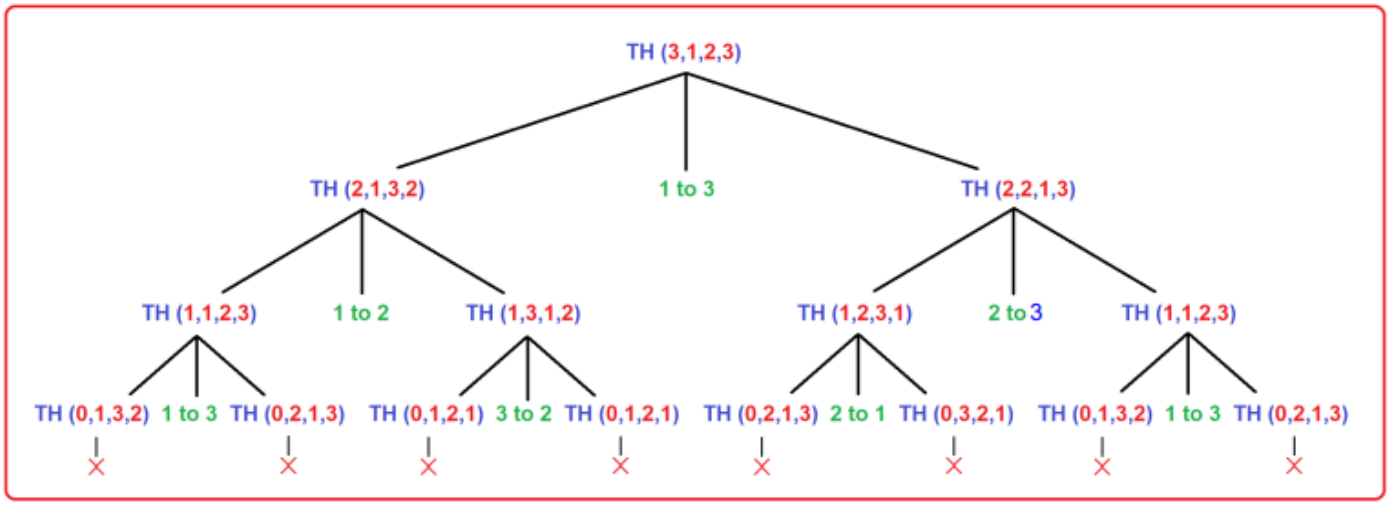
```
void TOH(int n,  int A,  int B,  int C)
{
        if(n>0)
        {
                TOH(n-1, A , C , B);
                printf( "Move a Disc from %d to %d", A , C);
                TOH(n-1, B , A , C);
        }
}
```

Tracing for 3 Discs



Printing will be done in the returning phase from left to right.





Move Number	Movement Direction	Description of the Movement
1	(1-3)	Move a disc from Tower 1 to Tower 3
2	(1-2)	Move a disc from Tower 1 to Tower 2
3	(3-2)	Move a disc from Tower 3 to Tower 2
4	(1-3)	Move a disc from Tower 1 to Tower 3

5	(2-1)	Move a disc from Tower 2 to Tower 1
6	(2-3)	Move a disc from Tower 2 to Tower 3
7	(1-3)	Move a disc from Tower 1 to Tower 3

Remember, for n discs, there will be minimum 2^n-1 moves.

```
#include <stdio.h>
#include <stdlib.h>
void towers_of_hanoi(int n, char *a, char*b, char *c);
int cnt=0;
int main(void)
{
   int n;
   printf("Enter number of discs: ");
   scanf("%d",&n);
   towers_of_hanoi(n, "Tower 1", "Tower 2", "Tower 3");
}
void towers_of_hanoi(int n, char *a, char*b, char *c)
{
   if(n ==1)
```

```
    {
        ++cnt;
        printf ("\n%5d: Move disk 1 from %s to %s", cnt,a,c);
        return;
    }
    else
    {
        towers_of_hanoi(n-1, a, c,b);
        ++cnt;
        printf ("\n%5d: Move disk %d from %s to %s",cnt, n,a, c);
        towers_of_hanoi(n-1, b, a,c);
        return;
    }
}
```

Output:

```
Enter number of discs: 3
    1: Move disk 1 from Tower 1 to Tower 3
    2: Move disk 2 from Tower 1 to Tower 2
    3: Move disk 1 from Tower 3 to Tower 2
    4: Move disk 3 from Tower 1 to Tower 3
    5: Move disk 1 from Tower 2 to Tower 1
```

6: Move disk 2 from Tower 2 to Tower 3

7: Move disk 1 from Tower 1 to Tower 3

Enter number of discs: 4

1: Move disk 1 from Tower 1 to Tower 2

2: Move disk 2 from Tower 1 to Tower 3

3: Move disk 1 from Tower 2 to Tower 3

4: Move disk 3 from Tower 1 to Tower 2

5: Move disk 1 from Tower 3 to Tower 1

6: Move disk 2 from Tower 3 to Tower 2

7: Move disk 1 from Tower 1 to Tower 2

8: Move disk 4 from Tower 1 to Tower 3

9: Move disk 1 from Tower 2 to Tower 3

10: Move disk 2 from Tower 2 to Tower 1

11: Move disk 1 from Tower 3 to Tower 1

12: Move disk 3 from Tower 2 to Tower 3

13: Move disk 1 from Tower 1 to Tower 2

14: Move disk 2 from Tower 1 to Tower 3

15: Move disk 1 from Tower 2 to Tower 3